Experimental Report

Submodular Optimization and Greedy Approximation for Distributed Influence Maximization

Yixin Chen Bichi Zhang Yaoqin Ye

CS240: Algorithm Design and Analysis, Spring 2026

Abstract

This project studies influence maximization, a classical network optimization problem that asks how to choose a small number of seed nodes so that a diffusion process reaches as many nodes as possible. Following the CS240 project requirements, we connect a modern data application to classical algorithms: monotone submodular maximization, greedy approximation, lazy evaluation, and a communication-constrained distributed variant. We implemented Independent Cascade simulation, random/degree/PageRank baselines, centralized Monte Carlo greedy, CELF lazy greedy, and a GreeDI-style local candidate sharing method. Experiments were run on both quick small benchmarks and a larger course-scale preset. The current results show that structural baselines are surprisingly strong under the weighted-cascade probability rule, that CELF preserves greedy quality while cutting large-preset spread evaluations from 1026.5 to 266.2 on average, and that distributed candidate sharing exposes a clear tradeoff among solution quality, runtime, and communication budget.

1 Background and Related Work

Influence maximization is a standard problem in social networks, viral marketing, public-health intervention, and misinformation analysis. Given a network and a limited budget, the goal is to choose seed nodes that maximize the expected number of eventually activated nodes. The problem is algorithmic rather than purely empirical: the chosen seed set must solve a combinatorial optimization problem under uncertainty.

Kempe, Kleinberg, and Tardos [1] introduced influence maximization under the Independent Cascade (IC) and Linear Threshold (LT) models. They showed that the problem is NP-hard, but that the expected spread function is monotone and submodular. Nemhauser, Wolsey, and Fisher's classical result [2] then implies that greedy maximization under a cardinality constraint obtains a $(1 - 1/e)$ approximation for the exact submodular objective.

The difficulty is that exact influence spread is expensive to compute. A straightforward implementation estimates the spread of many candidate seed sets using Monte Carlo simulation. Leskovec et al. [3] proposed CELF, a lazy greedy method that uses submodularity to avoid many repeated marginal-gain evaluations. For distributed settings, Mirzasoleiman et al. [4] proposed GreeDI, which runs greedy locally on partitions and then merges local representatives. Recent work reinforces that scalability is still an active algorithmic issue: TIM/IMM [5, 6] use reverse influence sampling, PaC-IM [7] compresses and parallelizes influence maximization sketches, Chen et al. [8] study distributed submodular maximization in MapReduce and adaptive-complexity models, and GreediRIS [9] combines reverse influence sampling with distributed streaming maximum coverage. In this project, these systems motivate the communication-constrained question, but the implementation focuses on a transparent GreeDI-style reproduction.

2 Problem Formulation

Let $G = (V, E, p)$ be a directed graph. Each edge (u, v) has an activation probability $p_{uv} \in [0, 1]$. Given a seed budget k, the influence maximization problem is

$$\max_{S \subseteq V,\ |S| \le k} \sigma(S),$$

where $\sigma(S)$ is the expected number of active nodes after diffusion starts from seed set S.

Our implementation uses the Independent Cascade model. When a node u first becomes active, it gets one chance to activate each inactive out-neighbor v with probability p_{uv}. Each activation attempt is independent, and the process stops when no newly active nodes remain. Under the IC model, σ is monotone and submodular:

$$\sigma(A \cup \{v\}) - \sigma(A) \ge \sigma(B \cup \{v\}) - \sigma(B), \quad A \subseteq B,\ v \notin B.$$

This diminishing-returns property is the reason greedy selection has a theoretical guarantee and the reason CELF can safely treat old marginal gains as upper bounds.

3 Representative Method Analysis

3.1 Centralized Greedy

The centralized greedy algorithm starts with $S_0 = \emptyset$. At round t, it chooses

$$v^\star = \arg\max_{v \in V \setminus S_t} \left(\widehat{\sigma}(S_t \cup \{v\}) - \widehat{\sigma}(S_t)\right),$$

where $\widehat{\sigma}$ is the Monte Carlo estimate of IC spread. The seed set is updated as $S_{t+1} = S_t \cup \{v^\star\}$ until $|S| = k$.

If the graph has n nodes and m edges, one IC simulation costs $O(n+m)$ in the worst case. With R Monte Carlo simulations per estimate, naive greedy costs approximately

$$O(knR(n + m)),$$

ignoring small savings from already selected nodes. This is feasible for small course-project graphs but becomes expensive quickly.

3.2 CELF Lazy Greedy

CELF maintains a priority queue of cached marginal gains. Since submodularity implies that a node's marginal gain cannot increase as the seed set grows, an old marginal gain is an upper bound on its current gain. When the top node in the queue was last evaluated for the current seed-set size, CELF accepts it. Otherwise, it recomputes only that node's marginal gain and pushes it back. The selected seed set targets the same greedy objective but typically requires far fewer spread evaluations.

3.3 GreeDI-Style Distributed Greedy

For the distributed component, the node set is partitioned into m local parts. Worker i runs greedy over its local candidate list and returns at most q candidates. The coordinator forms

$$C = \bigcup_{i=1}^{m} C_i$$

and runs a final greedy selection over C rather than all nodes. We test $m \in \{2, 4, 8\}$ and $q \in \{k, 2k, 5k\}$. The communication proxy is the number of transmitted candidates, approximately mq but capped by partition sizes.

4 Reproduction and Implementation

The source code is organized as follows:

- `src/simulation.py`: Independent Cascade simulation and Monte Carlo spread estimation.
- `src/baselines.py`: random, weighted out-degree, and PageRank seed selection.
- `src/greedy.py`: centralized greedy and CELF lazy greedy.
- `src/distributed.py`: random node partitioning and GreeDI-style candidate sharing.
- `src/graphs.py`: benchmark graph construction and weighted-cascade probability assignment.
- `src/experiments.py`: end-to-end experiment runner and CSV output.

Because exact reproduction of large influence maximization systems is beyond the scope of the course project, we reproduced the core algorithmic mechanisms: IC spread estimation, greedy submodular selection, CELF lazy evaluation, and distributed local-candidate merging. The final code supports two benchmark presets. The quick `small` preset uses Erdos–Renyi, Barabasi–Albert, and Watts–Strogatz graphs with $n \in \{40, 70\}$, plus Zachary's Karate Club graph with 34 nodes. The `large` preset uses the same three synthetic families with $n \in \{50, 100, 200\}$, plus Karate. The synthetic large graphs use $k = 10$, while Karate uses $k = 5$. The reported CSV files are stored in the output directory as `influence_results.csv` and `influence_results_large.csv`.

Figure 1 shows the graph structures used by the quick small preset. These plots are included because graph topology directly affects which seed-selection heuristics work well: Erdos–Renyi graphs emphasize random connectivity, Barabasi–Albert graphs contain hubs, Watts–Strogatz graphs emphasize local small-world structure, and Karate provides a small real social network.

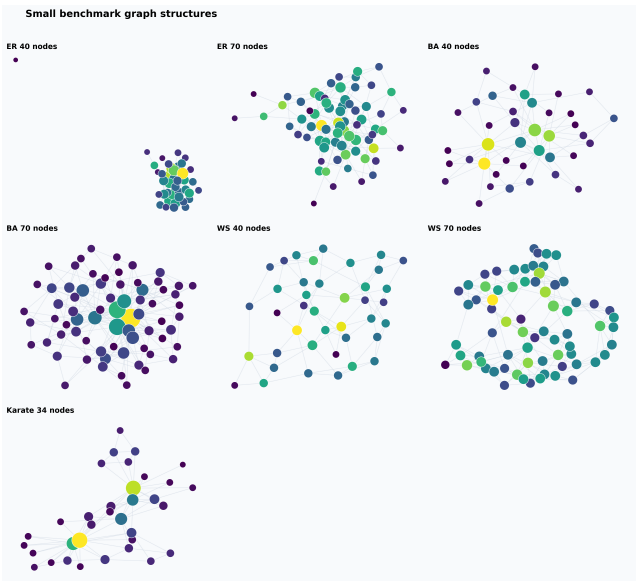

Figure 1: Small benchmark graph structures colored by weighted out-degree.

Each algorithm's final seed set was re-evaluated using a common Monte Carlo seed for the same graph case. This makes the reported spread more comparable than the internal spread estimates used during selection.

5 Experimental Evaluation

The report embeds the topology overview and one figure for each main metric: estimated spread, quality ratio, transmitted candidates, spread evaluations, and runtime. The experiment script also generates horizontal and vertical comparison bar charts under `figures/comparisons/`; those plots are not all embedded here because they restate the same metrics in a different layout.

5.1 Overall Algorithm Comparison

Table 1 summarizes the current large-preset results across ten graph cases. The reported ratio is relative to centralized greedy on the same graph after fair re-evaluation. The small preset remains available as a quick smoke test and shows the same qualitative pattern, but the large preset better reflects the final project state.

Table 1: Average performance across all benchmark cases.

Algorithm	Spread	Ratio	Runtime (s)	Evaluations
Random	31.59	0.79	0.00	1.0
Weighted degree	43.71	1.07	0.01	1.0
PageRank	35.19	0.89	0.00	1.0
Greedy	40.43	1.00	4.67	1026.5
CELF	40.05	0.99	0.77	266.2

The large-preset experiment still does not show a clean dominance of greedy over structural baselines. Weighted degree is especially strong because the weighted-cascade probability rule gives structurally central nodes high influence potential. This is not a contradiction of the

greedy approximation theorem: the theorem compares exact greedy to the optimal seed set for the exact submodular objective, while our implementation selects using noisy Monte Carlo estimates and then re-evaluates final seed sets with another random seed.
CELF behaves as expected. Figures 2 and 3 show that it obtains essentially the same average spread as greedy, with ratio 0.99, while reducing spread evaluations from 1026.5 to 266.2 and runtime from 4.67 seconds to 0.77 seconds on average. This is the clearest theory-to-practice result of the reproduction.

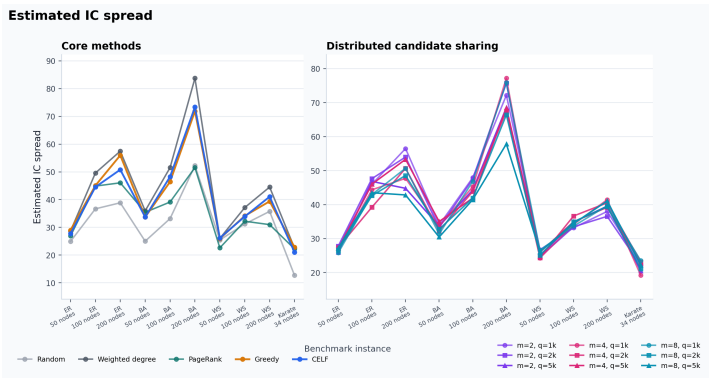

Figure 2: Estimated IC spread on the large preset.

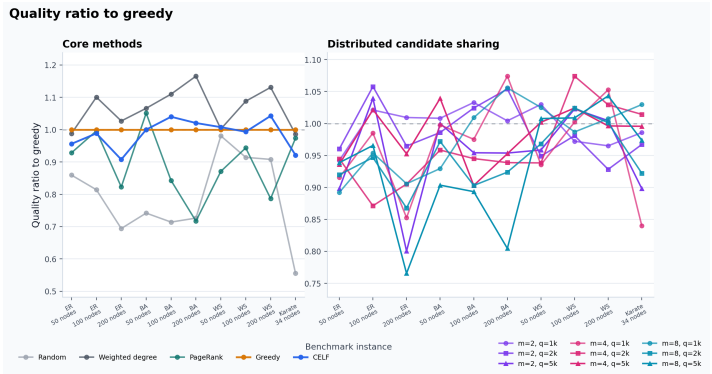

Figure 3: Quality ratio relative to centralized greedy on the large preset.

5.2 Distributed Communication Tradeoff

Table 2 summarizes the large-preset distributed variants. The notation $(m, q/k)$ means m partitions and local candidate budget $q = (q/k) \cdot k$.

Table 2: Average distributed performance under different partition and communication budgets.

Setting	Spread	Ratio	Runtime (s)	Evaluations	Candidates
$(2, 1)$	40.43	1.00	1.54	1130.5	19.0
$(2, 2)$	40.28	0.99	3.78	2114.5	38.0
$(2, 5)$	38.40	0.95	9.89	3984.1	78.4
$(4, 1)$	39.39	0.96	1.54	1234.5	38.0
$(4, 2)$	38.56	0.96	3.43	2072.1	66.4
$(4, 5)$	39.48	0.98	6.44	3068.1	108.4
$(8, 1)$	39.70	0.98	1.66	1362.1	66.4
$(8, 2)$	37.95	0.94	2.86	1924.9	96.4
$(8, 5)$	36.72	0.93	3.12	2080.9	108.4

Figure 4 shows the expected cost side of the tradeoff clearly: larger local budgets increase transmitted candidates and usually increase spread evaluations. The quality side is not monotone. The best average distributed setting in the large preset is $(m, q/k) = (2, 1)$, which is essentially tied with centralized greedy after fair re-evaluation. Increasing q does not automatically improve quality in this run because local and coordinator selection still rely on noisy Monte Carlo estimates. The right interpretation is that small candidate pools can be competitive, while extra communication has real computational cost and needs more stable estimation to be consistently useful.

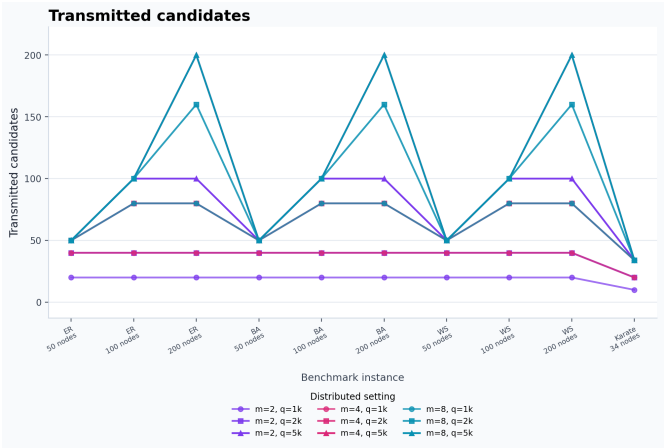

Figure 4: Communication proxy: number of transmitted candidates on the large preset.

5.3 Runtime and Evaluation Counts

Figures 5 and 6 show that centralized greedy becomes noticeably more expensive on the large preset: average spread evaluations rise to 1026.5. CELF reduces this to 266.2 by reusing marginal-gain upper bounds. Distributed variants perform more total evaluations because each worker runs a local greedy phase and the coordinator runs another greedy phase. This is acceptable only when worker computations are parallelized or when full-graph candidate search is impossible due to memory, ownership, privacy, or communication constraints. The current implementation models the communication-limited algorithmic structure, but it does not execute workers in parallel.

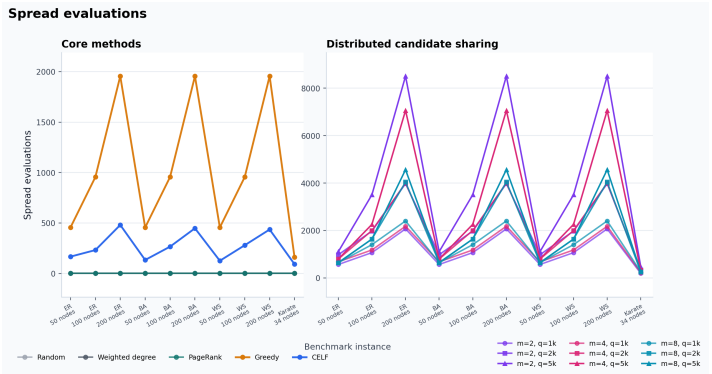

Figure 5: Number of spread evaluations on the large preset. CELF reduces evaluations compared with naive greedy.

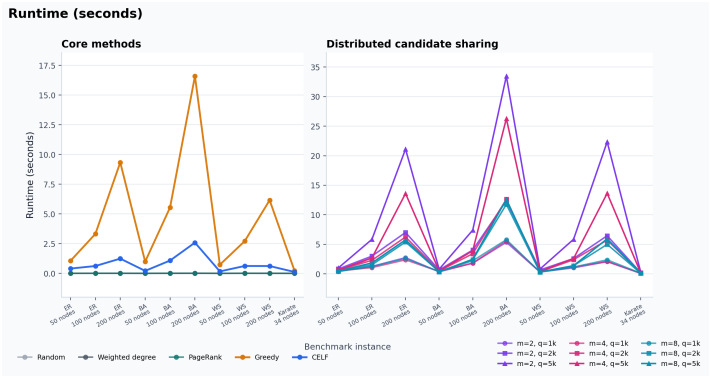

Figure 6: Wall-clock runtime on the large preset.

6 Deviations, Limitations, and Optional Extensions

The proposal mentioned larger synthetic graphs with $n \in \{500, 1000, 5000\}$ and possible real networks such as Wiki-Vote or NetHEPT. The final reproduction uses course-scale synthetic graphs up to 200 nodes plus Karate, so that the whole experiment can run reliably with Monte Carlo greedy in the course-project environment. This choice keeps the implementation transparent, but it limits strong scalability claims. The experiment runner now supports custom synthetic sizes through `--sizes`, but the submitted CSV outputs use the documented small and large presets.

Another limitation is Monte Carlo noise. The experiment uses a modest number of simulations for speed, and distributed variants use fewer simulations during their local selections. As a result, the selected seed set may be sensitive to randomness. This explains why fair re-evaluation sometimes ranks weighted degree, CELF, or distributed variants above centralized greedy. A stronger final experiment would increase the simulation count, average over multiple random seeds, and report confidence intervals.

The implemented optional extension is the GreeDI-style communication-constrained variant, together with small and large presets plus graph-structure visualizations. Further possible extensions include deterministic partition strategies, larger real datasets, RR-set based maximum coverage, and parallel worker execution to measure true distributed speedup.

7 Conclusion

This project demonstrates how a modern network problem can be understood using classical algorithmic ideas. Influence maximization under IC diffusion gives a monotone submodular objective; greedy selection provides a theoretical approximation guarantee; CELF turns submodularity into a practical acceleration; and GreeDI-style candidate sharing connects the problem to distributed optimization under communication limits. The empirical results are intentionally course-scale, but they reproduce the central algorithmic mechanisms and expose the main practical issue: influence maximization quality depends not only on the approximation algorithm, but also on spread-estimation accuracy, graph structure, and communication constraints.

Code Availability

The complete source code, experiment scripts, figures, and report sources for this project are publicly available at `https://github.com/Snake-Konginchrist/CS240_Course_Project`.

References

[1] D. Kempe, J. Kleinberg, and E. Tardos. Maximizing the spread of influence through a social network. *KDD*, 2003.

[2] G. L. Nemhauser, L. A. Wolsey, and M. L. Fisher. An analysis of approximations for maximizing submodular set functions. *Mathematical Programming*, 1978.

[3] J. Leskovec, A. Krause, C. Guestrin, C. Faloutsos, J. VanBriesen, and N. Glance. Cost-effective outbreak detection in networks. *KDD*, 2007.

[4] B. Mirzasoleiman, A. Karbasi, R. Sarkar, and A. Krause. Distributed submodular maximization: Identifying representative elements in massive data. *NeurIPS*, 2013.

[5] Y. Tang, X. Xiao, and Y. Shi. Influence maximization: Near-optimal time complexity meets practical efficiency. *SIGMOD*, 2014.

[6] Y. Tang, Y. Shi, and X. Xiao. Influence maximization in near-linear time: A martingale approach. *SIGMOD*, 2015.

[7] L. Wang, X. Ding, Y. Gu, and Y. Sun. Fast and space-efficient parallel algorithms for influence maximization. *PVLDB*, 2023.

[8] Y. Chen, T. Dey, and A. Kuhnle. Scalable distributed algorithms for size-constrained submodular maximization in the MapReduce and adaptive complexity models. *JAIR*, 2024.

[9] R. Barik, W. Cappa, S. M. Ferdous, M. Minutoli, M. Halappanavar, and A. Kalyanaraman. GreediRIS: Scalable influence maximization using distributed streaming maximum cover. *Journal of Parallel and Distributed Computing*, 2025.